LAB1: Computer Vision

Lab on Edge Detection & Image Segmentation

Objective:

- **To study and implement edge detection, thresholding, and morphological operations for identifying object boundaries and segmenting digital images.**
- **To analyze their effectiveness in noise removal, shape enhancement, and image refinement in computer vision applications.**

LAB: Computer Vision

Introduction to Color Image Processing?

Edge Detection:
Edge detection identifies boundaries in an image by detecting sudden changes in intensity, helping to highlight object outlines.

Image Segmentation:
Image segmentation divides an image into meaningful regions or objects based on pixel similarity such as intensity, color, or texture.

LAB: Computer Vision

Importance for Object Detection?

Edge detection and image segmentation are important for object detection because they help the computer understand where objects begin and end in an image. **Edge detection** extracts object boundaries, while image segmentation separates objects from the background, making it easier for **object detection** algorithms to locate, classify, and analyze objects accurately in computer vision systems.

LAB: Computer Vision

Edge Detection & Image Segmentation?

Edge detection:

- Sobel
- Prewitt
- Canny

Thresholding: Global, Adaptive

Morphological operations.

LAB: Computer Vision

Edge Detection & Image Segmentation

Edge Detection: *An edge is a sudden change in brightness*

- Represents object boundaries

Importance of Edges

- Helps identify shapes
- Used in segmentation
- Reduces data while preserving structure.

LAB: Computer Vision

Sobel & Prewitt Edge Detectors

Sobel Operator

- Detects horizontal & vertical edges
- Uses weighted 3×3 filters
- Slight noise reduction

LAB: Computer Vision

Sobel & Prewitt Edge Detectors

Prewitt Operator

- Similar to Sobel
- Equal weight to pixels
- Simpler but more noise-sensitive

Task1-Load a grayscale image and apply different edge detection operators.

1. Load an image in grayscale using OpenCV and display it in Jupyter Notebook.
2. Apply Sobel operator in the horizontal direction and display the result.
3. Apply Sobel operator in the vertical direction and display the result.
4. Apply Canny edge detection on the same image.
5. Compare the results of Sobel and Canny edge detection in 3–4 lines.

Task2-Use the Canny edge detector with different threshold values..

1. Load any real-world image (road, building, or object).
2. Apply Canny edge detection using threshold values (50, 100).
3. Apply Canny edge detection using threshold values (100, 200).
4. Display all outputs side by side.
5. Mention, which threshold setting gives better edge detection?